

given utterance. Each of the different expressions of the intent is a paraphrase and they all map to the same intent. Hence it is the job of the NLU to identify which specific intent a user utterance maps to. This can be mapped to a multi-class text classification problem. Given a piece of text, namely, the user utterance, and each of the intents defined by the dialogue designer as one of the class labels, the task of predicting the intent of a new utterance gets reduced to a standard text classification problem. However, while there are well-known techniques of text classification, the issue is the need for a large volume of labelled data to train the classifiers for intent identification.

Labelled data in the form of *<utterance, intent>* is needed for this task. This is an issue especially for domain-specific chatbots, where there isn't sufficient labelled data for each intent.

Prior to the introduction of automated conversational agents, in many of these domain-specific settings, human agents have been interacting with users and have been helping them with their task-specific queries. This can be through phone calls or chats. Hence many of these use cases typically have a large number of voice call recordings/chat transcripts of customer-agent conversations. One way of obtaining training data for intent classification is to annotate this corpus for labelling utterances with their respective intents. However, this requires considerable manual effort and is error-prone. Hence there has been work on using unsupervised learning techniques to learn intents for utterances from this unlabelled corpus. This task is known as automatic intent mining.

For now, let us assume that we have chat transcripts of the conversations and each of these transcripts represents a document in a document corpus. One way of approaching the task of automatic intent mining is to cluster the unlabelled conversational corpus. Each of the clusters can be examined manually to see if they represent a specific intent. However, while this solution appears straightforward, it is often not practical. Note that each of these chats can consist of many sentences and is typically a multi-turn conversation. Here is a brief example of a chat:

Agent: Good morning. My name is Frank.

User: Good morning Frank.

Agent: Thank you for contacting us. How may I assist you today?

User: I have submitted my motor insurance enrolment form a week back. However, I have not yet received the enrolment card.

Agent: Please wait a minute while I check out the details.

Agent: I see that the enrolment card was dispatched a couple of days back.

We find that the conversation contains some of the common courteous greetings that are not relevant to intent detection. These are followed by the user stating his query, followed by the agent's response. While this is a typical example, there can be many cases where the users state their requests/queries over multiple sentences with intervening

responses by the agent. Hence, identifying the user intent in such multi-turn conversations using clustering of the documents is not a straightforward task.

Given that most of the conversations would have a standard template for the exchange of courteous greetings between agent and user, the first step is to remove such common statements across all chats. We can then separate the agent and user utterances. We can use the heuristic that the users are likely to express their requests/queries in the first few sentences of their conversation with the agent. Hence, we will extract the first few sentences from the customer utterance and use that as the document on which clustering will be done.

There are a number of clustering algorithms we can try out to cluster our document corpus. For any clustering problem, there are important decisions one needs to make.

- What is the representation of the input?
- What is the distance metric we would use to compute the distance/separation between the input points?
- What is the clustering method to use?

Some of the clustering algorithms require that we specify the number of clusters before the clustering is done. K-means, a popular clustering algorithm, belongs to this category. You may be wondering how we can specify the number of clusters we need in advance for our task of automatic intent mining. Well, the answer is that we do not know this number. Hence, we need to try with different values of *k* (which is the number of clusters) for the k-means and manually inspect the results until we are satisfied with the results. There are other algorithms which do not require us to specify the number of clusters beforehand. DBSCAN, another popular clustering algorithm, belongs to this class. However it requires you to specify a different parameter ahead of clustering. Hence, in most cases, clustering is typically an iterative process. Next, we need to decide on the remaining two questions, namely: (a) input representation, and (b) distance metric. We will cover this in the next column.

I have a question for our readers. Given a set of questions, we want to solve two different tasks. The first task is finding similar questions and the second one is identifying duplicate questions. What would your approach be for each of these tasks? Think about your answer and look at how the solution can be applied to the problem of automatic intent mining. Which of the above two tasks is more relevant to the problem of intent recognition? We will discuss this topic in more detail in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Here's wishing all our readers happy coding until next month. 

 By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.